✈️ AeroTrack Pro — Project Review Cheat Sheet

STATUS: PRODUCTION READY

Smart Airport Baggage Tracking & Passenger Experience System

ACADEMIC CONTEXT
PSG College of Technology — MCA Project

LIVE ENVIRONMENT
Frontend :5173 · API :3001 · MongoDB Atlas

TEAM AUTHORS
Sanjiv Kumar A · Dheena Dhayal M · Rebakha Snowit M

1. Executive Summary & Technology Stack

AeroTrack Pro is a modular airport baggage tracking and operations platform. It features client-side QR generation and live camera QR tag scanning, responsive baggage journey tracking, and a secure Express authentication API connected directly to a cloud **MongoDB Atlas** database cluster.

LAYER	TECHNOLOGIES USED	RATIONALE & KEY ADVANTAGE
Frontend	Vanilla JavaScript (ES Modules), HTML5, CSS3	Fast rendering, zero runtime framework bundle overhead, native modular imports.
Dev & Bundler	Vite 7	High-speed HMR, production chunk splitting, automated same-origin `/api` proxying.
Backend API	Node.js 22, Express 5.1	Lightweight asynchronous I/O, native JSON body parsing, modern middleware chaining.
Database	MongoDB Atlas (Cloud M0 Cluster)	Cloud-managed high availability, flexible document model, native TTL index expiration.
Database Driver	Official `mongodb` native driver (v6.20)	Official native driver (`MongoClient`) delivering raw speed without Mongoose ODM overhead.
Security Engine	Node.js `crypto` (scrypt, HMAC, timingSafeEqual)	Salted memory-hard password hashing, timing-attack resistant OTP digest verification.
Quality Assurance	Vitest (43 unit tests), ESLint 9	JSDOM-based unit testing, strict zero-warning code style enforcement.

2. How MongoDB Atlas is Connected & Configured

The backend connects to MongoDB Atlas using the official Node.js driver `mongodb`. The connection string is managed securely via environment variables:

```
MONGODB_URI=mongodb+srv://sanjivvv19_db_user:<password>@aerotrack.7fv3sq4.mongodb.net/aerotrack?retryWrites=tr
MONGODB_DB=aerotrack
SESSION_SECRET=ff22643ca5e4d11478264ae82cf584ce4bf66e971da553601ec96293c514b1c717...
PORT=3001
EMAIL_PROVIDER=dev
```

Database Collections & Indexes:

COLLECTION	FIELDS STORED	INDEXES & AUTOMATED BEHAVIOR
`users`	`_id`, `name`, `email`, `passwordHash`, `role`, `emailVerified`, `preferences`, `createdAt`	**Unique index on** `email`. Passwords stored as `salt:scryptDerivedHex` (64-byte key).
`sessions`	`_id` (HMAC token digest), `userId`, `createdAt`, `expiresAt`	**TTL Index on** `expiresAt` (`expireAfterSeconds: 0`). MongoDB automatically purges expired sessions after 14 days.
`auth_challenges`	`email`, `purpose`, `codeHash`, `attempts`, `expiresAt`, `createdAt`	**Compound unique index** `{ email: 1, purpose: 1 }` + **TTL Index** (5-minute expiration for OTP codes).
`email_outbox`	`to`, `subject`, `text`, `createdAt`	**TTL Index** (7-day auto-cleanup) storing verification emails for dev inspection.

> **Resilient Fault-Tolerant Database Architecture**
> If Atlas rejects the connection due to an unwhitelisted dynamic IP (TLS alert 80) or network drop, the server **never crashes** (avoiding breaking the Vite dev runner). It activates an in-memory dev fallback and automatically reconnects to MongoDB Atlas once accessible.

3. Security Architecture & Cryptography

- **Password Protection**: Uses Node's asynchronous `crypto.scrypt` with a 16-byte cryptographically secure random salt and a 64-byte derived key. Resists GPU and dictionary attacks.
- **Zero Raw Cookie Storage (Hash-Before-Store)**: The browser receives an `HttpOnly` base64 session token, but the database only stores its HMAC-SHA256 digest (`tokenDigest`). If the database is compromised, active session cookies cannot be impersonated.
- **Timing-Attack Immunity**: Verification of OTP code hashes uses `crypto.timingSafeEqual` to ensure comparison time is constant regardless of where mismatch occurs.
- **Brute-Force & Abuse Defense**: 5-attempt limit per OTP challenge; 60-second cooldown timer between OTP resends; global rate limiter (150 req / 15 min) and auth rate limiter (20 req / 15 min).
- **CSRF & Protocol Defense**: Enforces custom `X-Requested-With: aerotrack` header on all state-changing API endpoints and caps JSON request body payloads at 16 KB.

4. Engineering Challenges Overcome During Development

CHALLENGE ENCOUNTERED	UNDERLYING ROOT CAUSE	ENGINEERING SOLUTION IMPLEMENTED
Vite Proxy Not Forwarding /api Calls	Running `vite client` set the root to `client/` . Vite looked for `client/vite.config.js` and ignored the root config file.	Added `client/vite.config.js` , updated `package.json` with explicit `--config vite.config.js` , and configured proxy target `http://127.0.0.1:3001` .
MongoDB Atlas TLS Alert 80 Failure	MongoDB Atlas security layer rejects TLS handshakes (SSL alert 80 / `ERR_SSL_TLSV1_ALERT_INTERNAL_ERROR`) when client IP is not in Atlas Network Access.	Whitelisted `0.0.0.0/0` in Atlas Network Access, added friendly TLS diagnostic detection, and built in-memory fallback auto-reconnection.
Server Crash Terminating Dev Runner	Concurrently used `-k` (kill-others). When DB connection threw an unhandled error, the entire dev server and frontend halted.	Wrapped DB startup in fault-tolerant retry loop so Express and Vite stay alive continuously in all network conditions.
Root Workspace Execution	Running `npm run dev` from top-level folder failed because `package.json` was inside subfolder `AeroTrack-` .	Created root-level forwarding `package.json` and unified `.env` configurations.

5. PSG College Team Contribution Breakdown

TEAMMATE	ASSIGNED MODULES	MAIN CONTRIBUTION & DELIVERABLES
Sanjiv Kumar A	`client/js/main.js`, `router.js`, `auth.js`, `layouts.js`, `pages/landing.js`, `pages/about.js`, `pages/auth.js`, `server/*`	Routing engine, MongoDB Atlas backend API, password security, session management, landing and authentication pages.
Dheena Dhayal M	`pages/passenger.js`, `components/baggage.js`, `components/lookup.js`, `shared/qr.js`, `shared/index.js`	Passenger baggage tracking pages, QR code tag generation, camera-based and image QR scanning lookup.
Rebakha Snowit M	`pages/staff.js`, `pages/board.js`, `pages/admin.js`, `pages/general.js`, `lib/api.js`, `shared/prediction.js`	Airport staff checkpoint operations, flight arrival board, operations analytics, and delay prediction logic.

6. Quick Code Reference Guide (Where to Point in Review)

FEATURE / CODE SNIPPET	FILE LOCATION	WHAT TO HIGHLIGHT TO REVIEWERS
scrypt Password Hashing	`server/index.js` (Lines ~95–109)	Point to `scrypt(password, salt, 64)` and `timingSafeEqual()`.
MongoDB TTL Session Index	`server/index.js` (Lines ~548–555)	Point to `expireAfterSeconds: 0` on the `expiresAt` index.
Health Check & Status API	`server/index.js` (Lines ~160–175)	Show `GET /api/health` returning database provider, dbMode, and uptime.
Reverse Proxy Routing	`vite.config.js`	Show `proxy: { '/api': 'http://127.0.0.1:3001' }`.
QR Tag Encoding & Canvas	`client/shared/qr.js`	Show payload encoding format `aerotrack://bag/<ref>`.
Unit Test Suite	`client/test/*.test.js`	Show 43 automated unit tests in Vitest covering router, DOM, domain, and baggage lookup.

7. Top Viva / Reviewer Questions & Model Answers

Q1: Why did you choose the official native `mongodb` driver instead of `mongoose`?

Answer: We chose the official `mongodb` native driver (`MongoClient`) because it eliminates Mongoose's internal schema compilation overhead and change-tracking memory footprint. It delivers raw JSON speed, clean native `async/await` operations (`insertOne`, `findOneAndUpdate`), and allows declaring database-level TTL and unique indexes directly on the database engine.

Q2: How do you handle password storage securely?

Answer: We never store plain text passwords. We use Node's native `crypto.scrypt` with a 16-byte cryptographically secure random salt and a 64-byte derived key. Because `scrypt` is memory-hard, it prevents dictionary and GPU hardware attacks. During login, we re-derive the hash using the stored salt and compare it using `crypto.timingSafeEqual`.

Q3: How does session management work across Vite and Express?

Answer: Upon successful authentication, Express issues an `HttpOnly; SameSite=Lax` session cookie containing a 32-byte base64 token. The database only stores an HMAC-SHA256 digest of that token. During local development, Vite proxies all `/api` calls to port 3001 while preserving same-origin cookie transmission.

Q4: How does MongoDB automatically clean up expired sessions?

Answer: We create a **TTL (Time-To-Live) index** on the `expiresAt` field with `expireAfterSeconds: 0`. MongoDB Atlas runs a background task that automatically finds documents whose `expiresAt` timestamp has passed and deletes them directly at the database layer without requiring backend cron jobs.